

```
!pip install pandas
import pandas as pd
import numpy as np
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/d
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (
```

```
print(pd.__version__)
```

```
2.2.2
```

```
s=pd.Series([1,2,3,4,5])
#from LIST
s=pd.Series([1,2,3,4,5], index=['a','b','c','d','e'])
#From Dictionary
student={
    "name":"sarika kerin",
    "branch":"bca"
}
se=pd.Series(student)
print(se)
```

```
name      sarika kerin
branch              bca
dtype: object
```

```
n=pd.Series([10,20,30,40,50])
```

```
n.mean()
```

```
np.float64(30.0)
```

```
n.max()
```

```
50
```

```
n.min()
```

```
10
```

```
n.describe()
```

```
      0
count  5.000000
mean  30.000000
std    15.811388
min    10.000000
25%    20.000000
50%    30.000000
75%    40.000000
max    50.000000
```

dtype: float64

```
#Boolean indexing
s[s > 3]
```

```
      0
d  4
e  5
```

dtype: int64

```
#Vectorized string operations
names = pd.Series(['nandu', 'harshi', 'bharathi'])
names.str.lower()
```

```
      0
0  nandu
1  harshi
2  bharathi
```

dtype: object

```
names.str.len()
```

```
      0
0  5
1  6
2  8
```

dtype: int64

```
names.str.startswith('A')
```

```

      0
0  False
1  False
2  False

```

dtype: bool

```
names.str.upper()
```

```

      0
0  NANDU
1  HARSHI
2  BHARATHI

```

dtype: object

```

student={
    "names":["harshi","nandu","bharu"],
    "branch":["aiml","aiml","aiml"]
}

```

```

s=pd.DataFrame(student)
print(s)

```

```

      names branch
0  harshi   aiml
1   nandu   aiml
2   bharu   aiml

```

```

std=[
    {"name":"harshi","branch":"aiml"},
    {"name":"nandu","branch":"aiml"},
    {"name":"bharu","branch":"aiml"},
]
s=pd.DataFrame(std)
print(s)

```

```

      name branch
0  harshi   aiml
1   nandu   aiml
2   bharu   aiml

```

```

arr = np.random.rand(3, 4)
c = pd.DataFrame(arr, columns=['A', 'B', 'C', 'D'])
print(c)

```

```

      A      B      C      D
0  0.804766  0.667722  0.924479  0.633944
1  0.370182  0.497188  0.004537  0.787374
2  0.534759  0.369604  0.208640  0.522284

```

```
# Exploring Data Frame
n=pd.Series([10,20,30,40,50])
```

Start coding or [generate](#) with AI.

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_3542/1442273385.py in <cell line: 0>()
----> 1 d.head()
```

NameError: name 'd' is not defined

Next steps: [Explain error](#)

```
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'San Francisco', 'Los Angeles', 'Chicago']
}
d=pd.DataFrame(data)
print(d)
```

	Name	Age	City
0	Alice	25	New York
1	Bob	30	San Francisco
2	Charlie	35	Los Angeles
3	David	40	Chicago

```
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'San Francisco', 'Los Angeles', 'Chicago']
}
d=pd.DataFrame(data)
```

```
d.head()
```

	Name	Age	City
0	Alice	25	New York
1	Bob	30	San Francisco
2	Charlie	35	Los Angeles
3	David	40	Chicago

Next steps: [Generate code with d](#) [New interactive sheet](#)

```
d.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
```

```
Data columns (total 3 columns):  
#   Column  Non-Null Count  Dtype  
---  ---  
0   Name    4 non-null      object  
1   Age      4 non-null      int64  
2   City     4 non-null      object  
dtypes: int64(1), object(2)  
memory usage: 228.0+ bytes
```

```
d.describe()
```

	Age 
count	4.000000
mean	32.500000
std	6.454972
min	25.000000
25%	28.750000
50%	32.500000
75%	36.250000
max	40.000000

```
d.columns
```

```
Index(['Name', 'Age', 'City'], dtype='object')
```

```
d.index
```

```
RangeIndex(start=0, stop=4, step=1)
```

```
d.dtypes
```

	0
Name	object
Age	int64
City	object

dtype: object

```
d.nunique()
```

```
      0
Name    4
Age     4
City     4

dtype: int64
```

```
d.isnull().sum()
```

```
      0
Name    0
Age     0
City     0

dtype: int64
```

```
d["Name"]
```

```
      Name
0    Alice
1     Bob
2   Charlie
3    David

dtype: object
```

```
d.Name
```

```
      Name
0    Alice
1     Bob
2   Charlie
3    David

dtype: object
```

```
d[['Name', 'Age']]
```

	Name	Age	
0	Alice	25	
1	Bob	30	
2	Charlie	35	
3	David	40	

```
d.loc[0]
```

	0
Name	Alice
Age	25
City	New York

dtype: object

```
d.iloc[0]
```

	0
Name	Alice
Age	25
City	New York

dtype: object

```
d.loc[0:2]
```

	Name	Age	City	
0	Alice	25	New York	
1	Bob	30	San Francisco	
2	Charlie	35	Los Angeles	

```
d.loc[0:2, 'Name':'City']
```

	Name	Age	City	
0	Alice	25	New York	
1	Bob	30	San Francisco	
2	Charlie	35	Los Angeles	

```
d.iloc[-1]
```

```
      3
Name    David
Age      40
City    Chicago
```

dtype: object

```
d.iloc[0:2,0:2]
```

```
   Name  Age
0  Alice  25
1   Bob   30
```

```
d.iloc[0, 0]
```

```
'Alice'
```

```
d.loc[0, 'Name']
```

```
'Alice'
```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

```
-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_3542/3087625485.py in <cell line: 0>()
----> 1 d.shape()
```

```
TypeError: 'tuple' object is not callable
```

Next steps: [Explain error](#)

```
n[n>20]
```



```
0
n[n<40]
```

```
3 40
```

```
0 10
```

```
1 20
dtype: int64
```

```
2 30
```

```
dtype: int64
```

```
s * 2
s+10
```

```
0
```

```
a 11
```

```
b 12
```

```
c 13
```

```
d 14
```

```
e 15
```

```
dtype: int64
```

```
print(n)
```